

گزارش فنی پروژه نويز خوانش چند کیوبیتی

هدف و تصویر کلی

هدف این گزارش، مستندسازی یک پایپلاین کامل برای مدل سازی نويز خوانش در یک سامانه چهار کیوبیتی است. پروژه از داده های خام تجربی IQ شروع می شود، از روی آن ها ماتریس های خطای خوانش را استخراج می کند، سپس این نویزها را روی مدارهای کوانتومی اعمال می کند و در نهایت با یک مدل یادگیری ماشین تلاش می کند توزیع احتمال نویزی را به توزیع ایده آل بازگرداند.

دو فایل اصلی پروژه چنین نقشی دارند:

- ماژول های خوانش داده ها استخراج ماتریس های نویز: بارگذاری داده های تجربی، آموزش تفکیک کننده IQ، استخراج ماتریس های 2×2 و 16×16 ؛
- ماژول های تولید و ذخیره دیتاست و مدل ها: ساخت مدار، اعمال سه نوع نویز، تولید دیتاست، آموزش MLP و ارزیابی با $L1$ fidelity و KL .

اجرای خط لوله (خلاصه):

۱. بررسی داده IQ؛
۲. استخراج ماتریس؛
۳. تولید دیتاست؛
۴. آموزش و ارزیابی در.

در نسخه فعلی پروژه، همه مسیرهای ورودی/خروجی به *snapshot* (مثلاً 1_1_2025) وابسته اند تا چند کمپین اندازه گیری تجربی بتوانند هم زمان نگهداری و مقایسه شوند.

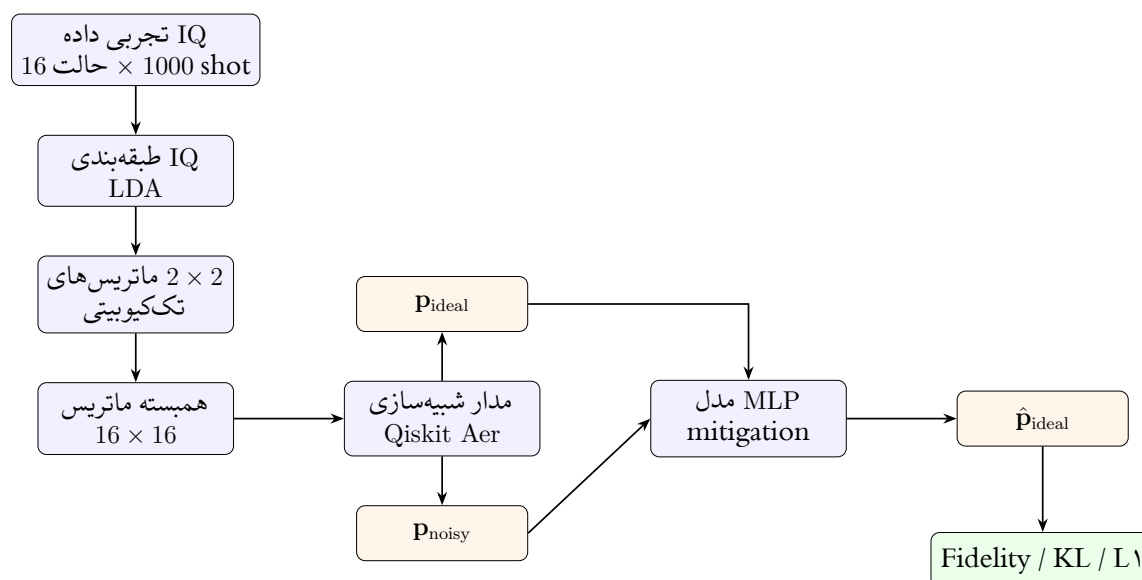
در کل پروژه یک قرارداد مهم حفظ می‌شود: ماتریس نویز خوانش به صورت

$$A_{ij} = P(\text{measured} = j \mid \text{prepared} = i) \quad (1)$$

تعریف شده است. بنابراین سطرها حالت آماده‌سازی و ستون‌ها حالت اندازه‌گیری هستند و هر سطر جمع یک دارد. این همان قرارداد مورد استفاده در Qiskit ReadoutError است.

معماری کلی خط لوله پروژه

پروژه یک مسیر پیوسته از داده خام تا ارزیابی mitigation دارد. شکل ۱ این مسیر را به صورت یک سیستم واحد نشان می‌دهد؛ نه چند آزمایش جدا.



شکل ۱: معماری کلی خط لوله: از داده IQ تا استخراج ماتریس نویز، تولید توزیع ایده‌آل/نویزی، -miti- gation با MLP و ارزیابی.

دو شاخه اصلی:

۱. شاخه کالیبراسیون: داده IQ → LDA → ماتریس‌های نویز؛

۲. شاخه mitigation: مدار کوانتومی $P_{ideal} \rightarrow P_{noisy}$ و $P_{ideal} \rightarrow P_{hat_{ideal}} \rightarrow MLP$.

شاخه اول «مدل نویز سخت‌افزار» را می‌سازد؛ شاخه دوم از آن برای تولید دیتاست و ارزیابی استفاده می‌کند.

ساختار snapshot و مسیرهای پروژه

برای توسعه به سمت چند مجموعه داده تجربی، خط لوله به صورت سلسله‌مراتبی سازماندهی شده است: هر اسنپ شات یک کمپین اندازه‌گیری مستقل است. ماتریس نویز استخراج شده از فقط روی دیتاست همان اسنپ شات اعمال می‌شود. این جداسازی برای مقایسه drift سخت‌افزار، افزودن داده جدید بدون بازنویسی کد، و تکرارپذیری آزمایش‌ها ضروری است.

داده‌های تجربی IQ

ساختار فایل‌ها

داده‌های تجربی در پوشه مربوطه قرار دارند. برای هر snapshot و برای هر یک از ۱۶ حالت پایه محاسباتی یک فایل وجود دارد؛ از 0000.txt تا 1111.txt. هر فایل چهار خط دارد؛ هر خط مربوط به یک کیوبیت است و شامل ۱۰۰۰ نمونه مختلط از نوع

$$z = I + iQ$$

است. بنابراین برای هر حالت آماده‌سازی، ۱۰۰۰ shot و برای کل پروژه

$$16 \times 1000 = 16000$$

نمونه چهارکیوبیتی داریم. پس از بارگذاری:

$$X \in \mathbb{C}^{16000 \times 4}, \quad Y \in \{0, 1\}^{16000 \times 4}.$$

در این نمایش، $X[n, q]$ نمونه IQ کیوبیت q در shot شماره n و $Y[n, q]$ بیت آماده‌سازی همان کیوبیت است.

برچسب‌گذاری و ترتیب بیت‌ها

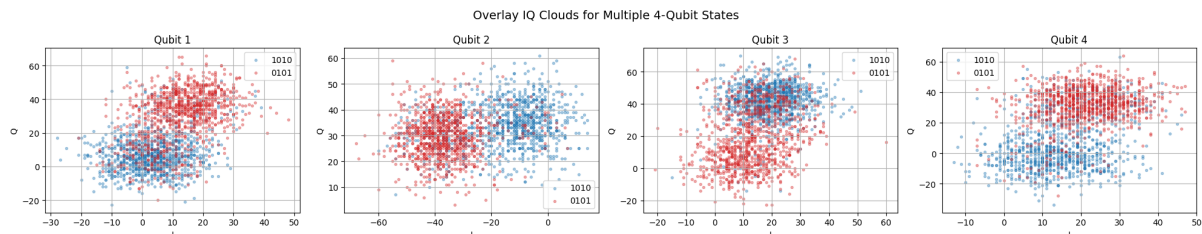
برچسب‌ها از نام فایل به دست می‌آیند. در کد، تابع `bits_from_filename` نام فایل را خوانده و برای سازگاری با ترتیب داخلی پروژه، بیت‌ها را معکوس می‌کند. قرارداد اندیس‌گذاری حالت‌ها چنین است:

$$i = b_1 + 2b_2 + 4b_3 + 8b_4. \quad (2)$$

پس b_1 کم‌ارزش‌ترین بیت است و b_4 پرازش‌ترین بیت. این قرارداد باید در همه بخش‌ها، از ساخت ماتریس نویز تا تبدیل شمارش‌های Qiskit به بردار احتمال، یکسان بماند.

ابرهای IQ و معنای فیزیکی آن‌ها

در اندازه‌گیری dispersive، خروجی هر shot یک نقطه در صفحه IQ است. اگر کیوبیت در حالت $|0\rangle$ آماده شده باشد، نقاط باید اطراف یک ابر و اگر در $|1\rangle$ آماده شده باشد اطراف ابر دیگری قرار گیرند. هرچه این دو ابر جداتر باشند، fidelity readout بالاتر است. همپوشانی ابرها همان ناحیه‌ای است که در آن تفکیک‌کننده ممکن است 0 را 1 یا 1 را 0 تشخیص دهد.



شکل ۲: ابرهای IQ برای دو حالت چهارکیوبیتی 0101 و 1010. هر پنل مربوط به یک کیوبیت است. جدایی و همپوشانی ابرها نشان می‌دهد خطای خوانش برای هر کیوبیت چقدر است و آیا crosstalk بین کیوبیت‌ها وجود دارد یا نه.

شکل ۲ چند نکته مهم را نشان می‌دهد. نخست، جدایی ابرها برای همه کیوبیت‌ها یکسان نیست، پس ماتریس‌های تک‌کیوبیتی نباید الزاماً مشابه باشند. دوم، شکل ابرها فقط تابع بیت همان کیوبیت نیست و حالت کلی چهارکیوبیتی هم می‌تواند مرکز یا پراکندگی ابر را تغییر دهد. این رفتار نشانه‌ای از crosstalk و دلیل اصلی نیاز به ماتریس همبسته 16×16 است.

استخراج ماتریس‌های نويز خوانش

مدل Linear Discriminant Analysis

در بخش اول پروژه، برای هر کیوبیت یک مدل Linear Discriminant Analysis یا LDA آموزش داده شده است. ورودی مدل برای کیوبیت q بردار دوبعدی

$$\mathbf{x} = (I, Q)^\top$$

و برچسب آن 0 یا 1 است. داده‌ها با `train_test_split` و نسبت 80/20 تقسیم می‌شوند و ویژگی‌ها قبل از آموزش با `StandardScaler` نرمال‌سازی می‌شوند.

LDA فرض می‌کند دو کلاس 0 و 1 در صفحه IQ تقریباً توزیع گاوسی با کواریانس مشترک دارند. برای کلاس k تابع امتیاز به صورت

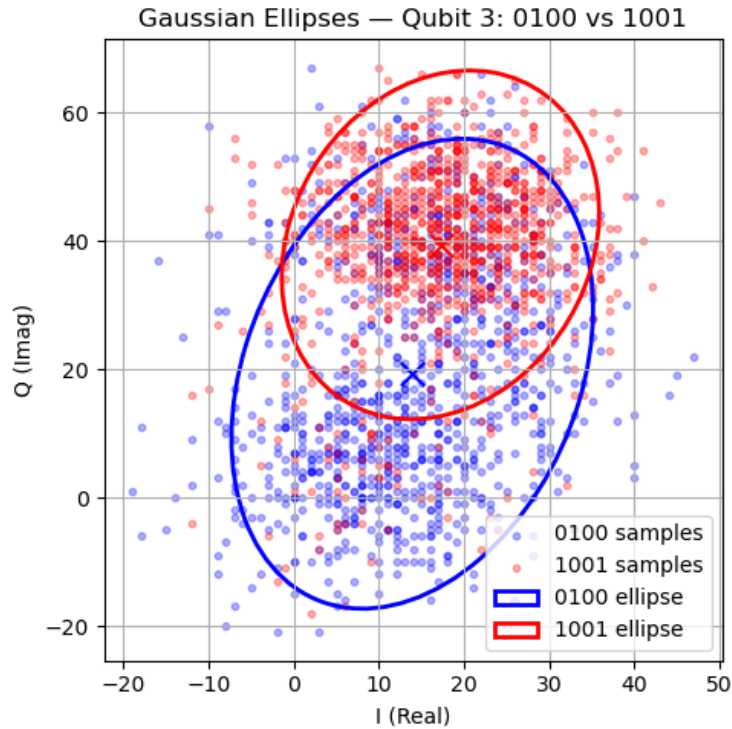
$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k \quad (3)$$

نوشته می‌شود. در این رابطه μ_k میانگین کلاس، Σ کواریانس مشترک و π_k prior کلاس است. مدل نمونه را به کلاسی نسبت می‌دهد که δ_k بزرگ‌تری دارد. برای دو کلاس، مرز تصمیم یک خط در صفحه IQ است:

$$\mathbf{w}^\top \mathbf{x} + w_0 = 0, \quad \mathbf{w} = \Sigma^{-1}(\mu_1 - \mu_0).$$

انتخاب LDA برای این پروژه منطقی است، چون مسئله تفکیک IQ دوبعدی است، مدل پایدار و قابل تفسیر است و هدف اصلی صرفاً بیشینه کردن accuracy نیست؛ هدف ساختن تخمینی پایدار از احتمال‌های $P(m|p)$ است. در نوت‌بوک، QDA نیز آزمایش شده است، اما LDA به عنوان گزینه ساده‌تر و پایدارتر انتخاب شده است.

شکل ۳ فرض توزیع گاوسی LDA را به صورت بصری تأیید می‌کند: هر حالت آماده‌سازی یک ابر متمرکز دارد، اما مراکز و پراکندگی‌ها برای حالت‌های مختلف چهارکیوبیتی متفاوت است — پیش‌زمینه‌ای برای نیاز به ماتریس همبسته 16×16 .



شکل ۳: بیضی‌های گاوسی برای کیوبیت ۳ در دو حالت آماده‌سازی 0100 (آبی) و 1001 (قرمز). محور افقی I و محور عمودی Q . همپوشانی ابرها و بیضی‌ها نشان می‌دهد چرا LDA در برخی ناحیه‌ها شات را اشتباه طبقه‌بندی می‌کند و fidelity خوانش از یک کاهش می‌یابد.

ماتریس‌های تک‌کیوبیتی 2×2

پس از آموزش مدل کیوبیت q ، خروجی مدل روی داده تست با برچسب واقعی مقایسه می‌شود. ماتریس تخصیص تک‌کیوبیتی از ماتریس confusion نرمال شده ساخته می‌شود:

$$A^{(q)} = \begin{pmatrix} P(m_q = 0 | p_q = 0) & P(m_q = 1 | p_q = 0) \\ P(m_q = 0 | p_q = 1) & P(m_q = 1 | p_q = 1) \end{pmatrix}. \quad (4)$$

در این ماتریس، عناصر قطری fidelity خوانش حالت‌های 0 و 1 هستند و عناصر غیرقطری احتمال خطای flip در خوانش را نشان می‌دهند. برای مثال، در یکی از اجراها ماتریس کیوبیت ۱ تقریباً چنین بوده است:

$$A^{(1)} \approx \begin{pmatrix} 0.794 & 0.206 \\ 0.369 & 0.631 \end{pmatrix}.$$

این یعنی وقتی کیوبیت ۱ در $|0\rangle$ آماده شده، حدود 20.6% احتمال دارد 1 خوانده شود، و وقتی در $|1\rangle$ آماده شده، حدود 36.9% احتمال دارد 0 خوانده شود.

ماتریس همبسته 16×16

برای ساخت ماتریس کامل، چهار مدل تک کیوبیتی روی کل دیتاست اعمال می شوند و برای هر shot یک بیت رشته پیش بینی شده $\hat{\mathbf{b}}$ به دست می آید. سپس برای هر حالت آماده سازی i و حالت اندازه گیری j شمارش انجام می شود:

$$M_{ij} = \frac{1}{N_i} \sum_{n: Y_n = \text{bits}(i)} \mathbb{I}[\text{bits_to_int}(\hat{\mathbf{b}}_n) = j]. \quad (5)$$

این ماتریس همان کانال کلاسیک خوانش چهار کیوبیتی است:

$$\mathbf{p}_{\text{measured}} = \mathbf{p}_{\text{prepared}} M.$$

اعتبارسنجی اصلی این است که برای هر سطر

$$\sum_{j=0}^{15} M_{ij} = 1.$$

در اجرای موجود، میانگین عناصر قطری این ماتریس حدود

$$\bar{F}_{\text{diag}} = \frac{1}{16} \sum_i M_{ii} \approx 0.385$$

بوده است. این عدد نشان می دهد خوانش چهار کیوبیتی دقیقاً به همان حالت آماده شده، به دلیل ضرب خطاهای چند کیوبیت و همبستگی ها، fidelity نسبتاً پایینی دارد.

۱.۰.۰ اعتبارسنجی ماتریس های نویز

استخراج ماتریس کافی نیست؛ باید مطمئن شویم خروجی یک کانال احتمال معتبر است. در غیر این صورت نمی توان آن را در شبیه سازی یا mitigation استفاده کرد.

بررسی های اصلی:

$$\begin{aligned}
 (6) \quad A_{ij} &\geq 0 && \text{(غیر منفی بودن)} \\
 (7) \quad \sum_j A_{ij} &= 1 && \text{(نرمال سازی سطری)} \\
 (8) \quad \bar{A}_{ii} &= \frac{1}{16} \sum_i A_{ii} && \text{(رفتار قطری)} \\
 (9) \quad M &\neq A^{(1)} \otimes A^{(2)} \otimes A^{(3)} \otimes A^{(4)} && \text{(عدم استقلال)}
 \end{aligned}$$

جدول ۱: چک لیست اعتبارسنجی ماتریس نویز

معنی فیزیکی	شرط	بررسی
احتمال منفی معنا ندارد	$A_{ij} \geq 0$	غیر منفی
هر حالت آماده به توزیع کامل می رود	$\sum_j A_{ij} = 1$	نرمال سازی
میانگین fidelity خوانش	$\bar{A}_{ii} \approx 0.39$	قطری
وجود correlation / crosstalk	$M \neq M_{\text{indep}}$	مقایسه

این اعتبارسنجی تضمین می کند ماتریس استخراج شده را می توان مستقیماً به عنوان کانال readout در تولید p_{noisy} استفاده کرد. اگر قرارداد سطری Qiskit رعایت نشود یا ordering بیت ها جابه جا شود، ماتریس از نظر ریاضی stochastic به نظر می رسد اما فیزیکاً اشتباه خواهد بود.

۲۰۰۰ چرا ماتریس 16×16 برابر کرونکر چهار ماتریس 2×2 نیست؟

اگر خطای خوانش چهار کیوبیت کاملاً مستقل بود، ماتریس کامل از کرونکر محصول ماتریس های تک کیوبیتی به دست می آمد:

$$M_{\text{indep}} = A^{(4)} \otimes A^{(3)} \otimes A^{(2)} \otimes A^{(1)}. \quad (10)$$

اما این فقط تحت فرض استقلال معتبر است. در داده تجربی، ابرهای IQ نشان می دهند حالت دیگر کیوبیت ها می تواند توزیع خوانش یک کیوبیت را جابه جا کند. بنابراین

$$M \neq M_{\text{indep}}$$

و همین اختلاف حامل اطلاعات فیزیکی مهم درباره crosstalk، خطاهای همبسته و غیرقابل فاکتورسازی بودن خوانش است.

به بیان دیگر، ماتریس‌های 2×2 پاسخ متوسط هر کیوبیت را جداگانه خلاصه می‌کنند، اما ماتریس 16×16 پاسخ کل register را به صورت مشترک ثبت می‌کند. انتظار برابری این دو فقط در یک مدل ایده‌آل مستقل درست است، نه در سخت‌افزار واقعی.

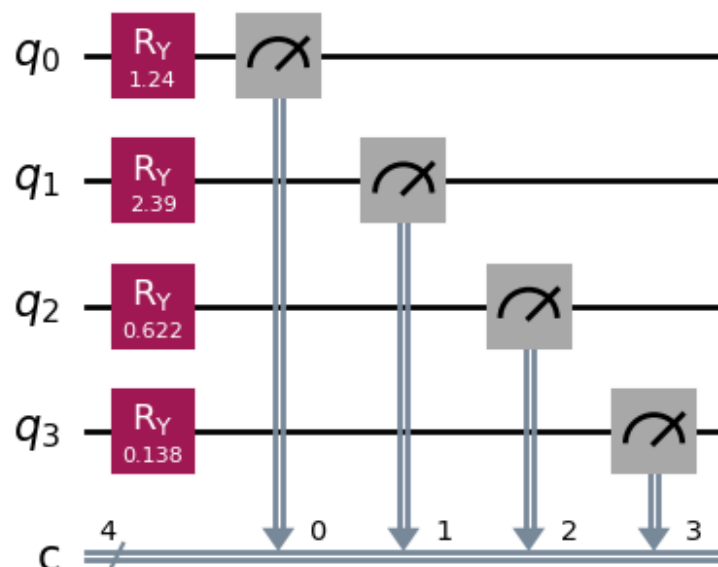
۱.۰ مدارهای کوانتومی استفاده‌شده در بخش دوم

در بخش دوم، از دو خانواده مدار برای تولید توزیع احتمال ایده‌آل و نویزی استفاده شده است. در هر نمونه، بردار پارامتر

$$\theta = (\theta_1, \theta_2, \theta_3, \theta_4)$$

به طور تصادفی از بازه $[0, \pi]^4$ انتخاب می‌شود.

۱.۱.۰ مدار مستقل RY Independent



شکل ۴: مدار مستقل شامل چهار گیت $R_y(\theta_i)$ و اندازه‌گیری نهایی. این مدار درهم‌تنیدگی ایجاد نمی‌کند و توزیع خروجی آن فاکتورسازی شده است.

در این مدار روی هر کیوبیت فقط یک گیت R_y اعمال می‌شود:

$$R_y(\theta) = e^{-i\theta Y/2}. \quad (۱۱)$$

اثر آن روی حالت اولیه $|0\rangle$ چنین است:

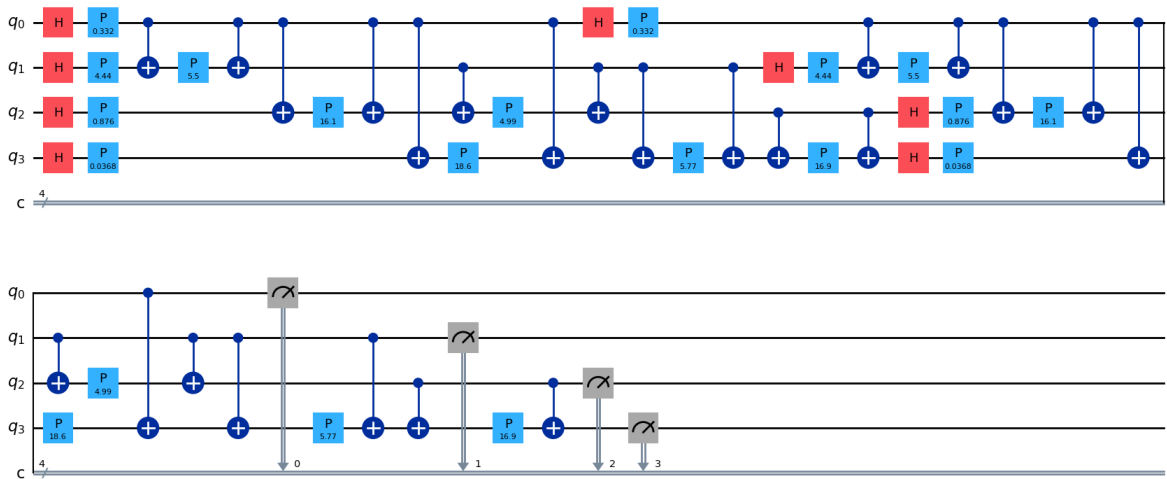
$$R_y(\theta)|0\rangle = \cos \frac{\theta}{2}|0\rangle + \sin \frac{\theta}{2}|1\rangle. \quad (۱۲)$$

پس احتمال اندازه‌گیری 1 برابر $\sin^2(\theta/2)$ و احتمال اندازه‌گیری 0 برابر $\cos^2(\theta/2)$ است. چون هیچ گیت دوکیوبیتی وجود ندارد، حالت کلی جدایی‌پذیر است و برای بیت‌رشته $s = s_1 s_2 s_3 s_4$:

$$P(s) = \prod_{k=1}^4 \left[\cos^2 \frac{\theta_k}{2} \right]^{1-s_k} \left[\sin^2 \frac{\theta_k}{2} \right]^{s_k}. \quad (۱۳)$$

این مدار benchmark ساده‌ای فراهم می‌کند: اگر mitigation روی این مدار هم خوب کار نکند، روی مدارهای پیچیده‌تر قابل اعتماد نخواهد بود.

۲.۱.۰ مدار ZZ FeatureMap



شکل ۵: مدار ZZFeatureMap چهارکیوبیتی. این مدار شامل لایه‌های هادامارد، گیت‌های فاز و بلوک‌های ZZ بین جفت کیوبیت‌هاست و توزیع احتمال غیرقابل فاکتورسازی تولید می‌کند.

مدار دوم با تابع `zz_feature_map` در Qiskit ساخته شده است. نقش گیت‌ها چنین است:

- **Hadamard**: حالت $|0\rangle$ را به $|+\rangle$ می‌برد و برهم‌نهی اولیه ایجاد می‌کند؛
- **gate Phase**: اطلاعات پارامتر θ_i را به فاز حالت‌ها وارد می‌کند؛
- **بلوک‌های ZZ**: برهم‌کنش‌های دوتایی از نوع $Z_i Z_j$ ایجاد می‌کنند و فاز حالت‌ها را به صورت وابسته به جفت ویژگی‌ها تغییر می‌دهند؛
- **Measurement**: توزیع احتمال روی ۱۶ حالت پایه را تولید می‌کند.

در پیاده‌سازی Qiskit، برهم‌کنش‌های ZZ معمولاً با الگوی CNOT—P—CNOT ساخته می‌شوند. این بلوک معادل یک فاز شرطی دوتایی است و باعث می‌شود توزیع خروجی دیگر به صورت ضرب چهار توزیع تک‌کیوبیتی نوشته نشود. به همین دلیل، ZZ FeatureMap تست سخت‌تری برای مدل mitigation است: هم مدار هم نویز می‌توانند همبستگی ایجاد کنند.

۳.۱.۰ نمونه کد: ساخت مدار ZZFeatureMap

در پروژه، مدار ZZ با تابع استاندارد `zz_feature_map` ساخته می‌شود و پارامترهای θ_i تزریق می‌شوند. نمونه زیر الگوی چهارکیوبیتی پروژه را نشان می‌دهد؛ برای مرور سریع بعدی، شناسه‌های اصلی در کامنت‌های انگلیسی توضیح داده شده‌اند.

```

1  # 4-qubit ZZFeatureMap circuit (same pattern as project dataset generation)
2  import numpy as np
3  from qiskit import QuantumCircuit
4  from qiskit.circuit.library import zz_feature_map
5  num_qubits = 4
6  thetas = np.array([0.3, 0.7, 1.1, 0.5]) # one angle per qubit / feature
7  # feature_dimension: number of qubits (feature variables)
8  # reps: how many times the feature-map block is repeated (circuit depth)
9  # entanglement: 'full' / 'linear' / 'circular' / explicit pair list
10 feature_map = zz_feature_map(
11     feature_dimension=num_qubits,
12     reps=2,
13     entanglement="full",)
14 qc = QuantumCircuit(num_qubits, num_qubits)
15 # bind angles and insert the feature-map sub-circuit

```

```

16 qc.compose(feature_map.assign_parameters(thetas), inplace=True)
17 qc.measure(range(num_qubits), range(num_qubits))
18 # qc is ready for ideal or noisy simulation (NoiseModel on AerSimulator)

```

در این پروژه reps=2 برای ایجاد تنوع کافی در توزیع خروجی انتخاب شده است (بخش ۴). حالت entanglement="full" یعنی برهم‌کنش ZZ بین همه جفت کیوبیت‌ها در هر لایه فعال است.

سه مدل نویز خوانش

نویز مصنوعی

در حالت synthetic، یک ماتریس flip ساده برای همه کیوبیت‌ها استفاده می‌شود:

$$A_{\text{syn}} = \begin{pmatrix} 1 - p_{01} & p_{01} \\ p_{10} & 1 - p_{10} \end{pmatrix}. \quad (۱۴)$$

در تولید دیتاست، p_{01} و p_{10} معمولاً از بازه [0.05, 0.15] انتخاب می‌شوند. در Qiskit Aer این نویز با ReadoutError و NoiseModel روی هر کیوبیت اعمال می‌شود.

نمونه کد: NoiseModel و ReadoutError در Aer

برای نویزهای synthetic و experimental_single، خطای خوانش تک‌کیوبیتی با ReadoutError به NoiseModel اضافه می‌شود و شبیه‌سازی نویزی از طریق AerSimulator انجام می‌گیرد. نمونه دو کیوبیتی زیر قرارداد پروژه را نشان می‌دهد: ماتریس A سطری است ($A_{ij} = P(m = j | p = i)$) و به هر کیوبیت جداگانه وصل می‌شود.

```

1 # Minimal 2-qubit example: independent per-qubit readout noise via Qiskit Aer
2 import numpy as np
3 from qiskit import QuantumCircuit, transpile
4 from qiskit_aer import AerSimulator
5 from qiskit_aer.noise import NoiseModel, ReadoutError
6 # Row-stochastic assignment matrix: A[i,j] = P(measured=j | prepared=i)
7 A_q0 = np.array([[0.92, 0.08], # prepared |0>

```

```

8         [0.11, 0.89]]) # prepared |1>
9 A_q1 = np.array([[0.90, 0.10],
10                 [0.12, 0.88]])
11 noise_model = NoiseModel()
12 noise_model.add_readout_error(ReadoutError(A_q0), [0]) # attach to qubit 0
13 noise_model.add_readout_error(ReadoutError(A_q1), [1]) # attach to qubit 1
14 # Simple entangled circuit
15 qc = QuantumCircuit(2, 2)
16 qc.h(0)
17 qc.cx(0, 1)
18 qc.measure([0, 1], [0, 1])
19 # Noisy simulation: readout errors applied at measurement time
20 sim = AerSimulator(noise_model=noise_model)
21 tqc = transpile(qc, sim)
22 result = sim.run(tqc, shots=4096).result()
23 counts = result.get_counts() # bitstring -> shot count

```

در پروژه چهارکیوبیتی، همین الگو در یک حلقه تکرار می‌شود. توجه: ماتریس همبسته 16×16 با این روش قابل اعمال مستقیم نیست (بخش بعد).

نویز تجربی تک کیوبیتی

در حالت `experimental_single`، چهار ماتریس $A^{(q)}$ که در بخش اول استخراج شده‌اند بارگذاری می‌شوند و برای هر کیوبیت به صورت مستقل به `NoiseModel` داده می‌شوند (الگوی کد در بخش ۳.۱.۰). این روش خطای واقعی هر کیوبیت را لحاظ می‌کند، اما همچنان فرض می‌کند کانال کلی فاکتورسازی شده است.

نویز تجربی همبسته

در حالت `experimental_correlated`، از ماتریس کامل M استفاده می‌شود. چون Qiskit Aer برای این کاربرد، اعمال مستقیم ماتریس همبسته 16×16 را به صورت `error readout` چندکیوبیتی در جریان

شبیه‌سازی استفاده نکرده است، نویز به شکل پس‌پردازش روی بردار احتمال اعمال می‌شود:

$$\mathbf{p}_{\text{noisy}} = \mathbf{p}_{\text{ideal}} M. \quad (۱۵)$$

این رابطه دقیقاً با قرارداد سطری ماتریس سازگار است: هر مؤلفه ایده‌آل p_i بین خروجی‌های اندازه‌گیری z طبق سطر M_{ij} پخش می‌شود.

تولید دیتاست، استراتژی آزمایش و مدل یادگیری ماشین

استراتژی طراحی دیتاست‌ها

ساختار پوشه‌های DataTrain فقط یک سازماندهی فایل نیست؛ پشت آن یک منطق آزمایشی مشخص قرار دارد. در نسخه نهایی پروژه، نویز آموزش و آزمون منطبق در نظر گرفته می‌شود: هر مدل فقط روی همان کانال نویزی ارزیابی می‌شود که با آن آموزش دیده است. این انتخاب باعث می‌شود تحلیل نتایج مستقیم‌تر و از نظر فیزیکی معنادارتر باشد.

جدول ۲: منطق طراحی دیتاست در فازهای مختلف (ارزیابی با نویز منطبق)

Phase	Train	Validation	Test (matched)
0	—	—	Independent/ZZ × 3 noises (baseline)
1	Ind. + Synthetic	Ind. + Synthetic	Ind. + Synthetic
2_0	Ind. + Exp. Single	Ind. + Exp. Single	Ind. + Exp. Single
2_1	Ind. + Exp. Corr.	Ind. + Exp. Corr.	Ind. + Exp. Corr.
3_0	ZZ + Synthetic	ZZ + Synthetic	ZZ + Synthetic
3_1	ZZ + Exp. Single	ZZ + Exp. Single	ZZ + Exp. Single
3_2	ZZ + Exp. Corr.	ZZ + Exp. Corr.	ZZ + Exp. Corr.

منطق ارزیابی:

۱. فاز ۰: (baseline) مدل هویت؛ ارزیابی روی هر سه نویز برای سنجش شدت خام نویز؛

۲. فازهای ۱ تا ۳: آموزش و آزمون روی همان مدار و همان نویز؛

۳. مقایسه نهایی: برای هر (نویز، مدار)، baseline در برابر مدل آموزش دیده.

خط لوله تولید دیتاست

برای هر نمونه دیتاست:

۱. بردار زاویه‌ها θ تصادفی تولید می‌شود؛
۲. مدار انتخاب‌شده ساخته می‌شود؛
۳. شبیه‌سازی بدون نویز انجام می‌شود و p_{ideal} به دست می‌آید؛
۴. یکی از سه نویز اعمال می‌شود و p_{noisy} تولید می‌شود؛
۵. ورودی و خروجی یادگیری ماشین ذخیره می‌شوند:

$$X = p_{\text{noisy}}, \quad Y = p_{\text{ideal}}.$$

هر بردار احتمال ۱۶ بعدی است و ترتیب مؤلفه‌ها مطابق حالت‌های ۰۰۰۰ تا ۱۱۱۱ است. دیتاست‌ها در ساختار مناسب ذخیره می‌شوند. در اجرای اصلی snapshot 1_1_2025، اندازه‌ها برای `valida- train` و `test` به ترتیب ۴۵۰۰، ۱۵۰۰ و ۱۵۰۰ نمونه بوده است.

نقش مدل MLP: تقریب کانال معکوس

مدل یادگیری ماشین یک شبکه کاملاً متصل است:

$$16 \rightarrow 128 \rightarrow 256 \rightarrow 128 \rightarrow 16.$$

لایه خروجی Softmax دارد تا $\hat{p}$ یک بردار احتمال معتبر باشد.

مدل MLP چیست و چه نیست:

- classifier حالت $|s\rangle$ نیست؛
- discriminator state روی IQ نیست؛
- تقریب غیرخطی از *channel readout inverse* هست:

$$f_{\theta} \approx \mathcal{M}^{-1}, \quad \hat{p}_{\text{ideal}} = f_{\theta}(p_{\text{noisy}}).$$

مدل M^{-1} را به صورت صریح محاسبه نمی‌کند (ماتریس 16×16 لزوماً معکوس پذیر نیست). در عوض، یک نگاشت غیرخطی $\mathbb{R}^{16} \rightarrow \mathbb{R}^{16}$ یاد می‌گیرد که توزیع نویزی را به توزیع ایده‌آل نزدیک کند. از نظر علمی، این learned readout error mitigation است.

آموزش با Adam ($lr = 10^{-3}$)، size batch ۶۴ و حدود epoch ۵۱ انجام شده است. زیان اصلی KLDivLoss است؛ در فاز 3_2 از MSELoss نیز استفاده شده است.

معیارهای ارزیابی و هاب‌لاین

۴.۱.۰ Fidelity (وفاداری / شباهت Bhattacharyya-based)

برای دو توزیع احتمال p (ایده‌آل) و q (پیش‌بینی یا نویزی)، fidelity کلاسیک پروژه:

$$F(p, q) = \left(\sum_{i=0}^{15} \sqrt{p_i q_i} \right)^2 \quad (۱۶)$$

تعریف شده است. این کمیت معیار شباهت است نه فاصله: مقدار آن بین 0 و 1 است؛ $F = 1$ یعنی دو توزیع کاملاً یکسان‌اند. در فیزیک کوانتومی fidelity برای مقایسه حالت‌ها و توزیع‌های اندازه‌گیری بسیار رایج است.

از دید فیزیکی، fidelity میزان همپوشانی دو توزیع را می‌سنجد. اگر نویز خوانش بخشی از احتمال را از حالت‌های درست به حالت‌های اشتباه منتقل کند، همپوشی کم می‌شود. اگر مدل mitigation موفق باشد، $F(p_{\text{ideal}}, \hat{p})$ افزایش می‌یابد.

L1 Error (فاصله منهن / Total Variation-like)

$$L_1(p, q) = \sum_{i=0}^{15} |p_i - q_i| \quad (۱۷)$$

این معیار متقارن و ساده است: مجموع اختلاف مطلق بین دو توزیع. تفسیر مستقیم دارد: «در مجموع چقدر جرم احتمال جابه‌جا شده؟» ارتباط نزدیک با Total Variation Distance دارد ($TV = \frac{1}{2} L_1$).

هرچه L_1 کوچک‌تر باشد، بازسازی بهتر است. برخلاف bounded fidelity در $[0, 1]$ نیست (حداکثر 2 برای بردارهای احتمال).

Divergence KL (واگرایی کولباک-لیبلر)

$$D_{KL}(\mathbf{p} \parallel \mathbf{q}) = \sum_{i=0}^{15} p_i \log \frac{p_i}{q_i} \quad (18)$$

این معیار می‌پرسد: «اگر توزیع واقعی $\mathbf{p}$ باشد و ما آن را با $\mathbf{q}$ تقریب بزنیم، چقدر اطلاعات از دست می‌دهیم؟» ویژگی‌های مهم:

- نامتقارن است: $D_{KL}(\mathbf{p} \parallel \mathbf{q}) \neq D_{KL}(\mathbf{q} \parallel \mathbf{p})$ ؛
 - وقتی q_i خیلی کوچک و p_i بزرگ باشد، مقدار به شدت رشد می‌کند؛
 - در نواحی با p_i بالا به تفاوت‌های کوچک حساس است.
- در این پروژه KLDivLoss برای آموزش MLP به کار رفته است، اما برای گزارش نهایی fidelity و L1 خواناترند.

نقش هر معیار در پروژه

جدول ۳: خلاصه معیارهای ارزیابی

معیار	نوع	بهرتر	کاربرد در پروژه
Fidelity	شباهت	بزرگ‌تر	گزارش نهایی، baseline مقایسه فازها
L1	فاصله	کوچک‌تر	خطای مطلق توزیع، تفسیر فیزیکی
KL	واگرایی	کوچک‌تر	زیان آموزش (KLDivLoss)

accuracy فقط می‌گوید بیشینه احتمال درست بوده یا نه؛ اما خروجی این پروژه یک توزیع کامل ۱۶ بعدی است. fidelity و L1 کل شکل توزیع را مقایسه می‌کنند.

سه حالت مقایسه در ارزیابی

هدف پروژه «بالا بردن fidelity به هر قیمتی» نیست؛ هدف کاهش اثر نویز خوانش است. سه حالت زیر همیشه باید با هم مقایسه شوند:

$$F_{\text{raw}} = F(\mathbf{p}_{\text{ideal}}, \mathbf{p}_{\text{noisy}}) \quad \text{حالت ۱ — بدون mitigation} \quad (19)$$

$$F_{\text{MLP}} = F(\mathbf{p}_{\text{ideal}}, \hat{\mathbf{p}}) \quad \text{حالت ۲ — پس از MLP} \quad (20)$$

$$\Delta F = F_{\text{MLP}} - F_{\text{raw}} \quad \text{حالت ۳ — بهبود} \quad (21)$$

اگر F_{raw} از ابتدا نزدیک ۱ باشد (مثلاً 0.972 در ZZFeatureMap اولیه)، ΔF کوچک لزوماً به معنای مدل ضعیف نیست؛ ممکن است نویز از ابتدا کم اثر بوده باشد. بنابراین همیشه باید baseline خام و بهبود نسبی را با هم گزارش کرد.

چالش‌ها، اصلاحات و نتایج

چالش‌های پروژه و راه‌حل‌ها

پروژه فقط اجرای کد نبود؛ چند مشکل فنی در مسیر حل شد:

جدول ۴: چالش‌های اصلی و اصلاحات انجام‌شده

چالش	مشکل	راه‌حل
ترتیب بیت‌ها	جاب‌جایی سطر/ستون M	یکسان‌سازی bits_to_int در همه بخش‌ها
قرارداد Qiskit	transpose اشتباه ماتریس	$A_{ij} = P(m = j p = i)$ اعمال $\mathbf{p}M$
ماتریس 16×16 ZZFeatureMap	Aer کانال همبسته کامل ندارد $F_{\text{raw}} \approx 0.972$ ، نویز کم اثر	apply_correlated_readout_noise افزایش reps به ۲
یکنواخت نویز train/test ساختار flat	ارزیابی cross-noise پیچیده یک snapshot، مسیرهای ثابت	ارزیابی منطبق (matched) معماری CLI + snapshot

بررسی چالش یکنواختی ZZFeatureMap

در فرآیند تولید داده برای ارزیابی عملکرد مدل یادگیری، یکی از چالش‌های مهم مربوط به استفاده از مدار ZZFeatureMap مشاهده شد. در مراحل اولیه، داده‌های مربوط به این مدار با پارامترهای تصادفی و تعداد لایه کم ($\text{reps}=1$) تولید شدند. بررسی اولیه نشان داد که شباهت بین توزیع ایده‌آل و توزیع نویزی حتی پیش از اعمال مدل یادگیری بسیار زیاد است. به عنوان نمونه، مقدار فیدیلیتی خام بین داده‌های ایده‌آل و نویزی به صورت زیر مشاهده شد:

$$F_{\text{raw}} \approx 0.972 \quad (22)$$

این مقدار نشان می‌دهد که ورودی مدل (توزیع نویزی) و خروجی هدف (توزیع ایده‌آل) از ابتدا اختلاف بسیار کمی دارند. بنابراین اگر پس از آموزش مدل، مقدار فیدیلیتی بالایی مانند $F \approx 0.996$ مشاهده شود، این نتیجه الزاماً نشان‌دهنده توانایی بالای مدل در یادگیری تصحیح نویز نیست، زیرا بخشی از این عملکرد ناشی از شباهت اولیه داده‌ها است.

بررسی توزیع‌های تولید شده نشان داد که دلیل اصلی این رفتار، ماهیت خروجی مدار ZZFeatureMap با تنظیمات اولیه بود. این مدار در حالت $\text{reps} = 1$ برای بسیاری از انتخاب‌های تصادفی پارامترها، توزیع احتمال حالت‌های اندازه‌گیری شده‌ای نزدیک به توزیع یکنواخت ایجاد می‌کند. در چنین شرایطی، اثر نویز خوانش روی توزیع احتمال کاهش پیدا می‌کند، زیرا تغییرات ایجاد شده توسط نویز در مقایسه با توزیع تقریباً یکنواخت بسیار کوچک است.

به بیان دیگر، مسئله اصلی در این مرحله، عدم صحت مدل نویز یا فرآیند اعمال ماتریس نویز نبود؛ بلکه داده‌های تولید شده از مدار، ساختار کافی برای آشکار کردن اثر نویز را نداشتند. در نتیجه مدل یادگیری ماشین عملاً با مسئله‌ای مواجه نبود که نیاز به اصلاح جدی داشته باشد.

برای رفع این مشکل، ساختار مدار تغییر داده شد و با افزایش عمق مدار از طریق افزایش تعداد لایه‌ها ($\text{reps} = 2$) تنوع بیشتری در توزیع‌های خروجی ایجاد شد. هدف از این تغییر، ایجاد یک تعادل مناسب بین دو عامل بود:

- وجود ساختار کافی در توزیع احتمال خروجی مدار، به طوری که مدل بتواند الگوهای کوانتومی را یاد بگیرد؛
- ایجاد اختلاف قابل مشاهده بین توزیع ایده‌آل و نویزی، به طوری که اثر نویز خوانش در داده‌ها باقی

بماند.

پس از اصلاح مدار، مقدار خلوص توزیع‌های تولید شده افزایش یافت و در عین حال توزیع‌ها از حالت تقریباً یکنواخت خارج شدند. این تغییر باعث شد داده‌های تولید شده برای آموزش و ارزیابی مدل، چالش واقعی‌تری ایجاد کنند و عملکرد مدل در شرایط مختلف نویز قابل تفسیرتر باشد.

لازم به ذکر است که هدف در این پروژه، بیشینه کردن یا کمینه کردن مستقیم معیارهایی مانند Purity یا Fidelity نیست؛ بلکه هدف اصلی ایجاد دیتاست‌هایی است که در آن‌ها:

۱. مدارهای کوانتومی دارای ساختار قابل یادگیری باشند؛

۲. نویز خوانش بتواند تغییر قابل اندازه‌گیری در توزیع احتمال ایجاد کند؛

۳. مدل یادگیری مجبور به استخراج نگاشت بین توزیع نویزی و توزیع ایده‌آل باشد.

بنابراین اصلاح پارامترهای مدار ZZFeatureMap به عنوان یک مرحله ضروری در آماده‌سازی داده انجام شد تا ارزیابی مدل در مراحل بعدی پروژه، بازتاب واقعی‌تری از توانایی مدل در کاهش اثر نویز کوانتومی داشته باشد.

جدول اصلی نتایج (ارزیابی با نویز منطبق)

در ارزیابی نهایی، هر مدل آموزش‌دیده فقط روی همان نویزی آزمون می‌شود که با آن آموزش دیده است؛ آزمون روی نویزهای نامرتب انجام نمی‌شود. برای هر جفت (نویز، مدار)، baseline از مدل هویت (بدون آموزش) و ستون Trained از فاز متناظر به دست می‌آید. جدول ۵ خلاصه میانگین fidelity روی ۱۵۰۰ نمونه test را نشان می‌دهد.

جدول ۵: میانگین fidelity روی test با نویز منطبق. train/test.

ΔF	Trained	Baseline	Noise	Circuit
۰۷۲.۰+	۹۷۹.۰	۹۰۷.۰	Synthetic	Independent
۲۰۰.۰+	۹۶۸.۰	۷۶۸.۰	Single Exp.	
۲۲۱.۰+	۹۸۴.۰	۷۶۳.۰	Correlated Exp.	
۰۳۵.۰+	۹۷۳.۰	۹۳۸.۰	Synthetic	FeatureMap ZZ
۰۸۵.۰+	۹۳۹.۰	۸۵۳.۰	Single Exp.	
۱۱۳.۰+	۹۷۴.۰	۸۶۱.۰	Correlated Exp.	

تحلیل baseline

ردیف Baseline نشان می‌دهد نویز خوانش، پیش از mitigation، چقدر توزیع ایده‌آل را خراب می‌کند. برای مدار مستقل، fidelity نویز مصنوعی 0.907 است، اما برای نویزهای تجربی به حدود 0.76–0.77 افت می‌کند؛ یعنی نویز تجربی (تک‌کیوبیتی یا همبسته) حدود 14–15 درصد همپوشانی بیشتری با ایده‌آل از بین می‌برد. برای ZZ FeatureMap ها baseline بالاترند (0.85–0.94)؛ شکل توزیع این مدار با همان نویزها در میانگین overlap بیشتری حفظ می‌کند، اما همچنان فضای قابل توجهی برای بهبود باقی می‌ماند.

اثر آموزش با نویز منطبق

وقتی train و test روی همان کانال نویز باشند، MLP به‌طور پایدار fidelity را بالا می‌برد:

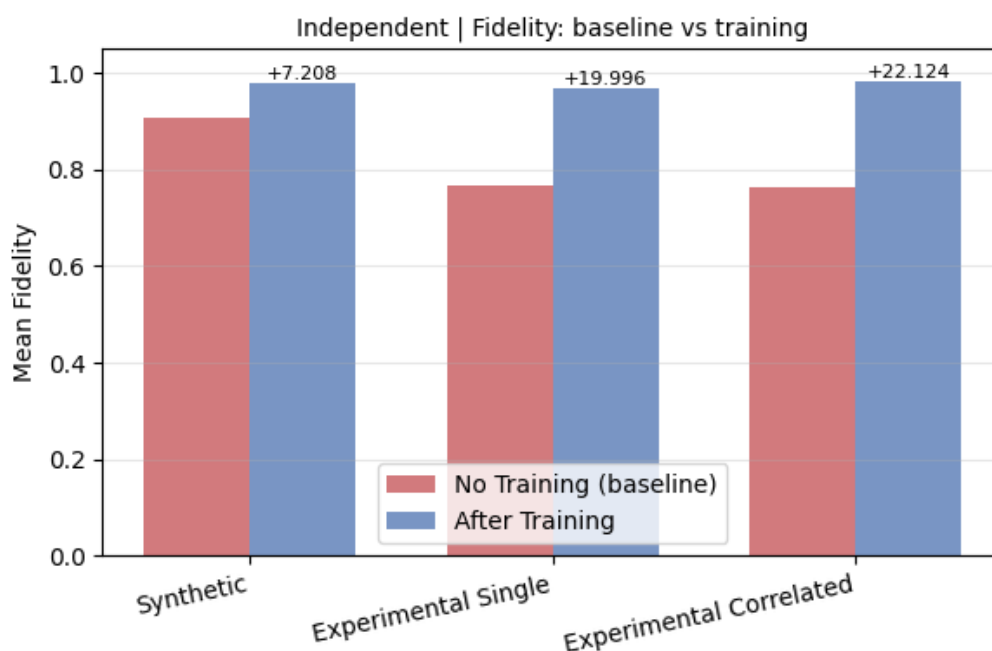
- Independent/Synthetic: $0.907 \rightarrow 0.979$ ($\Delta F = +0.072$)
- Independent/Exp. Single: $0.768 \rightarrow 0.968$ ($\Delta F = +0.200$)
- Independent/Exp. Correlated: $0.763 \rightarrow 0.984$ ($\Delta F = +0.221$)
- ZZ/Synthetic: $0.938 \rightarrow 0.973$ ($\Delta F = +0.035$)
- ZZ/Exp. Single: $0.853 \rightarrow 0.939$ ($\Delta F = +0.085$)
- ZZ/Exp. Correlated: $0.861 \rightarrow 0.974$ ($\Delta F = +0.113$).

بیشترین بهبود مطلق در مدار مستقل و نویز همبسته رخ داده است ($\Delta F \approx 0.22$): جایی که base-line پایین‌تر بوده و مدل بیشترین فضا برای بازگرداندن توزیع داشته است. در ZZ FeatureMap، چون baseline از ابتدا بالاتر است، ΔF کوچک‌تر است اما fidelity نهایی در هر سه نویز بالای 0.93 باقی می‌ماند.

نمودار baseline fidelity: در برابر آموزش

شکل‌های ۶ و ۷ خروجی تابع plot_fidelity_bars هستند و برای هر مدار، سه جفت میله (بدون آموزش / پس از آموزش منطبق) را نشان می‌دهند.

برای مدار مستقل (شکل ۶)، میله‌های قرمز (baseline) در نویزهای تجربی به‌وضوح پایین‌تر از نویز مصنوعی‌اند. پس از آموزش، هر سه میله آبی به ناحیه $F > 0.96$ می‌رسند. بیشترین فاصله بین baseline و



شکل ۶: مدار مستقل: مقایسه میانگین fidelity قبل و بعد از آموزش منطبق برای سه نوع نویز.

trained مربوط به Experimental Correlated است ($\Delta F \approx 0.22$)؛ این نشان می‌دهد مدل حتی برای کانال نویز 16×16 همبسته نیز نگاشت معکوس مؤثری یاد گرفته است.

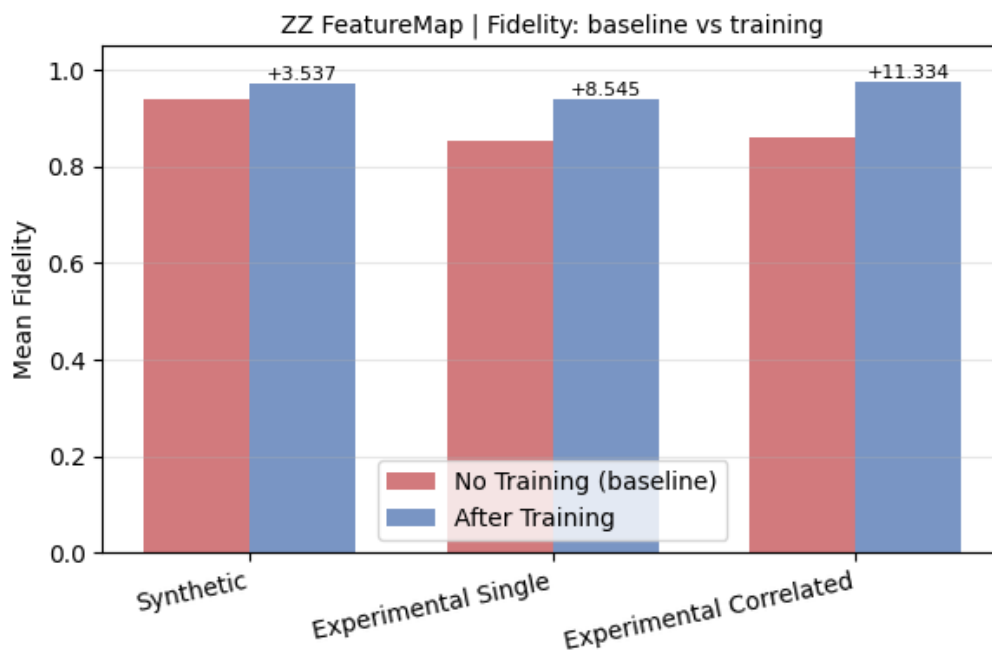
در ZZ FeatureMap (شکل ۷)، baseline نویز مصنوعی از ابتدا نزدیک 0.94 است؛ بنابراین سود آموزش محدودتر ($\Delta F \approx 0.035$) ولی همچنان معنادار است. برای نویزهای تجربی، بهبود به ترتیب +0.085 و +0.113 است و fidelity نهایی به 0.97 نزدیک می‌شود. این الگو با اصلاح reps در مدار سازگار است: داده‌ها چالش کافی دارند و مدل همچنان قادر به mitigation است.

داشبورد سه معیار

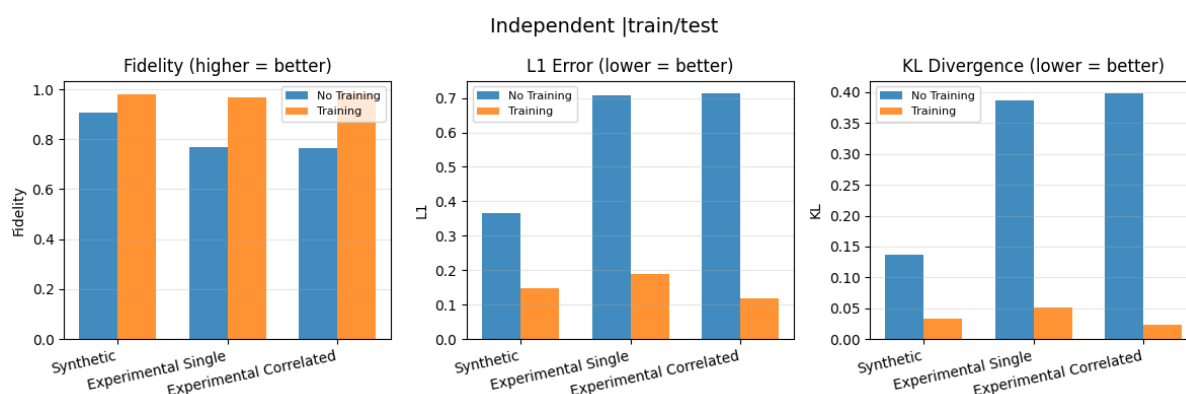
شکل‌های ۸ و ۹ خروجی matched_metrics_dashboard هستند و علاوه بر KL و L1 fidelity را در همان چارچوب matched گزارش می‌کنند.

در مدار مستقل (شکل ۸)، هر سه معیار یک داستان یکسان روایت می‌کنند: نویز تجربی baseline بدتری دارد (fidelity پایین، L1 و KL بالا) و آموزش هر سه را به‌طور همزمان بهبود می‌دهد. برای Experimental Correlated، L1 از حدود 0.72 به 0.12 و KL از حدود 0.40 به 0.02 کاهش می‌یابد؛ یعنی شکل کامل توزیع ۱۶ حالت بازسازی شده، نه فقط بیشینه احتمال.

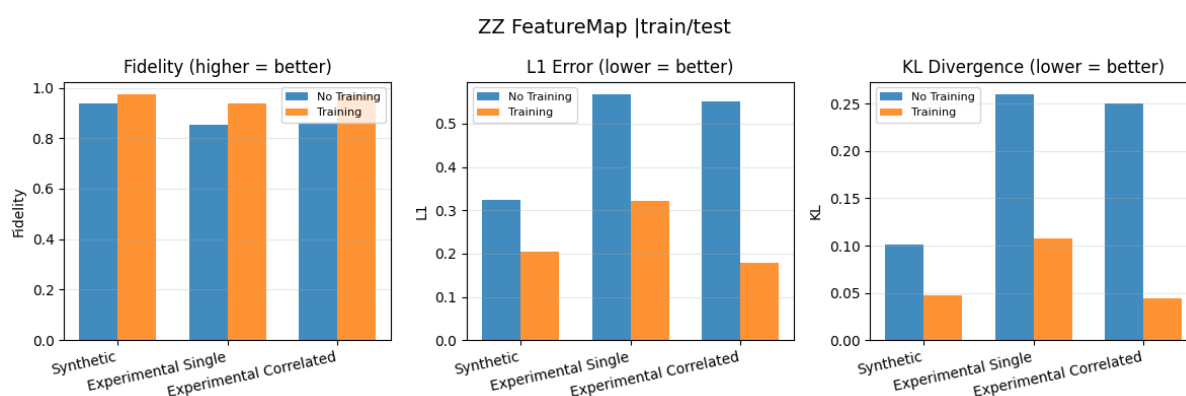
در ZZ FeatureMap (شکل ۹)، الگوی مشابهی دیده می‌شود با دامنه متفاوت: L1 baseline برای



شکل ۷: مدار ZZ FeatureMap: مقایسه میانگین fidelity قبل و بعد از آموزش منطبق.



شکل ۸: مدار مستقل: داشبورد سه معیار (fidelity، L1، KL)؛ مقایسه بدون آموزش و پس از آموزش منطبق.

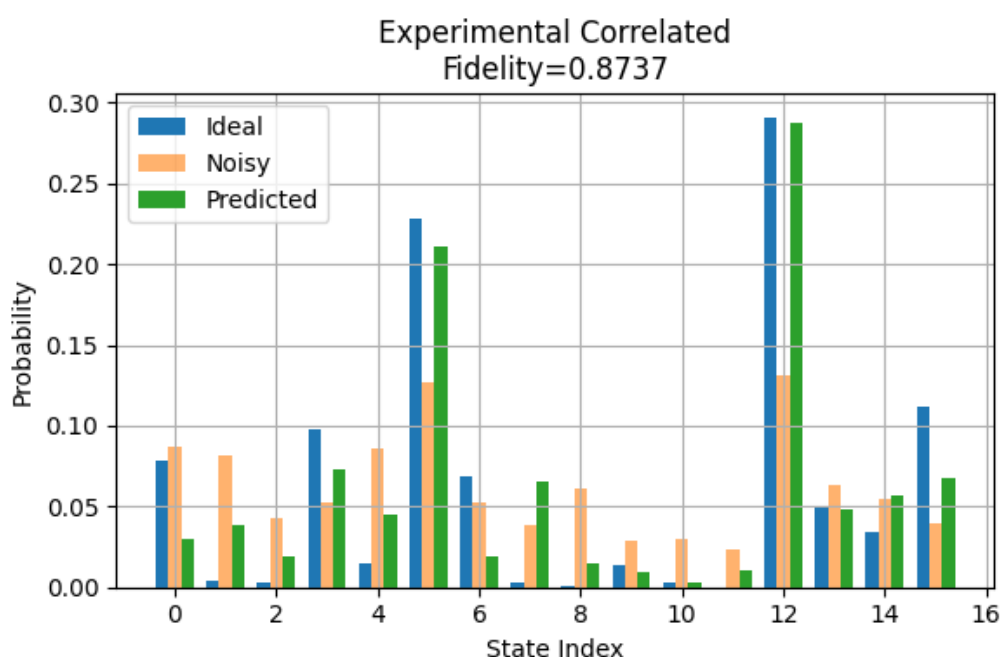


شکل ۹: مدار ZZ FeatureMap: داشبورد سه معیار در حالت ارزیابی منطبق.

نویزهای تجربی حدود 0.55–0.57 است و پس از آموزش به 0.18–0.32 می‌رسد. KL نیز در همه حالت‌ها به زیر 0.12 کاهش می‌یابد. نکته مهم این است که بهترین KL و L1 نهایی مربوط به Experimental Correlated است؛ در حالی که بیشترین ΔF در fidelity مربوط به همان نویز است — هم‌راستایی معیارها اعتبار نتایج را تقویت می‌کند.

نمونه تک‌نمونه‌ای بازسازی توزیع

جدول ۵ و نمودارهای تجمیعی، میانگین روی ۱۵۰۰ نمونه test را گزارش می‌کنند. برای درک بصری آنچه مدل انجام می‌دهد، تابع visualize_aligned_samples روی یک نمونه test، سه توزیع ۱۶ حالتی را کنار هم رسم می‌کند: ایده‌آل (Ideal)، نویزی (Noisy)، و خروجی MLP (Predicted). شکل ۱۰ خروجی این دستور برای فاز 2_3 است. یعنی مدار ZZ FeatureMap، نویز آموزش/آزمون Experimental Correlated، و یک نمونه تصادفی از دیتاست test.



شکل ۱۰: بازسازی توزیع برای یک نمونه test: ZZ FeatureMap با نویز همبسته تجربی. میله‌های آبی ایده‌آل، نارنجی نویزی، سبز پیش‌بینی MLP. fidelity این نمونه $F \approx 0.87$.

در این نمونه، بیشینه احتمال روی حالت‌های ۱۲ و ۵ متمرکز است. نویز خوانش بخشی از جرم احتمال را از این حالت‌ها به حالت‌های دیگر منتقل کرده (میله‌های نارنجی پهن‌تر و کوتاه‌تر از آبی). خروجی MLP (سبز) در اکثر اندیس‌ها به میله آبی نزدیک‌تر است؛ به‌ویژه در حالت ۱۲ که بیشینه اصلی است، پیش‌بینی تقریباً ایده‌آل را باز می‌گرداند. این همان چیزی است که شکل مفهومی؟؟ به صورت شماتیک نشان می‌داد،

اما اینجا روی داده واقعی test و کانال نویز 16×16 همبسته است. fidelity نمونه (≈ 0.87) از میانگین کل (0.974 در جدول) پایین تر است که طبیعی است: برخی نمونه ها سخت تر از میانگین بازسازی می شوند.

جمع بندی نتایج

۱. **mitigation مؤثر است:** در هر شش سناریو، fidelity پس از آموزش از baseline بالاتر است و KL/L1 کاهش می یابد.

۲. **نویز تجربی سخت تر است:** baseline پایین تر از نویز مصنوعی؛ بیشترین ΔF در مدار مستقل.

۳. **ارزیابی منطبق شفاف است:** هر سطر جدول ۵ یک سوال مشخص را پاسخ می دهد: «آیا مدل آموزش دیده روی نویز X ، توزیع را بهتر از مدل هویت باز می گرداند؟»

۴. **مدار ZZ baseline بالاتر، بهبود نسبی کمتر:** ΔF کوچک تر لزوماً ضعف نیست؛ fidelity نهایی همچنان بالای 0.93 است.

به روز رسانی زیر ساخت برای توسعه و چند snapshot

در طول این پروژه، علاوه بر نتایج علمی، زیر ساخت نرم افزاری نیز باز طراحی شد تا بتوانیم چند کمپین اندازه گیری تجربی را بدون تکرار دستی آزمایش ها مدیریت کنیم. انگیزه اصلی این کار، آماده سازی مسیر توسعه به سمت داده بیشتر، مقایسه drift سخت افزار، و در نهایت تبدیل این گزارش به مقاله قابل چاپ است.

چرا این تغییرات برای بهبود نتایج مؤثر است؟

۱. **داده تجربی بیشتر:** هر snapshot جدید (مثلاً روز یا تنظیمات کالیبراسیون متفاوت) بدون شکستن کد قبلی اضافه می شود؛ می توان mitigation را روی چند ماتریس نویز مقایسه کرد.

۲. **هم تراز فیزیکی:** ماتریس نویز و دیتاست ML همیشه از یک snapshot می آیند؛ خطای ناشی از جفت کردن اشتباه ماتریس و داده حذف می شود.

۳. **تکرار پذیری:** دستورات `run_readout_matrix.py` و `build_ml_datasets.py` خط لوله را مستند و قابل اجرای batch می کنند — الزام مقالات علمی.

۴. مقیاس‌پذیری: با افزایش تعداد، snapshot می‌توان مدل mitigation را روی داده ترکیبی یا چند سناریو drift ارزیابی کرد و robustness را سنجید.

۵. جداسازی: **concern** استخراج نویز (Part ۱) از تولید دیتاست و آموزش (Part ۲) جدا شده؛ هر فاز مستقل قابل اجرا و debug است.

چارچوب پیشنهادی برای مقاله

ساختار این گزارش عمداً با بخش‌بندی یک مقاله علمی هم‌راستا است:

جدول ۶: نگاشت بخش‌های گزارش به ساختار مقاله

بخش گزارش	معادل در مقاله
داده IQ و LDA	cali- Readout & setup Experimental Methods: bration
ماتریس 2×2 و 16×16	validation & extraction matrix Noise Methods:
مدارها و سه نوع نویز	models noise & families Circuit Methods:
استراتژی فازها و دیتاست	protocol training & design Dataset Methods:
جدول ۵ و نمودارها	performance mitigation Quantitative Results:
محدودیت‌ها و توسعه snap-shot	work Future & Discussion

گام بعدی برای انتشار: افزودن های snapshot بیشتر، تحلیل آماری اطمینان (مثلاً bootstrap روی test)، و در صورت امکان مقایسه با روش‌های کلاسیک mitigation (مثلاً معکوس ماتریس منظم‌شده).

محدودیت‌های فعلی پروژه

۱. مقیاس: فقط ۴ کیوبیت بررسی شده؛ برای $n > 4$ اندازه ماتریس $2^n \times 2^n$ به سرعت رشد می‌کند.

۲. وابستگی به کالیبراسیون: ماتریس‌های نویز وابسته به snapshot فعلی (1_1_2025) هستند؛ معماری جدید امکان افزودن های snapshot بعدی را فراهم کرده، اما در این گزارش فقط یک snapshot تحلیل شده است.

۳. ارزیابی منطبق: در این گزارش فقط حالت matched train/test بررسی شده؛ تعمیم به نویز یا مدار دیگر در scope فعلی نیست.

۴. تقریب غیرخطی: MLP معکوس دقیق M نیست؛ distribution outside ممکن است ضعیف عمل کند.

۵. نویز فقط readout هست: gate error و decoherence در شبیه‌سازی لحاظ نشده‌اند.

بیان صریح این محدودیت‌ها اعتبار گزارش را بالا می‌برد و مسیر ادامه کار را روشن می‌کند.

نکات حساس و توصیه‌های عملی

۱. قرارداد سطری ماتریس: همیشه $A_{ij} = P(m = j | p = i)$ است. بنابراین اعمال نویز همبسته به

صورت $p_{ideal}M$ انجام می‌شود.

۲. ماتریس 16×16 را با کرونکر اشتباه نگیریم: کرونکر فقط مدل مستقل است؛ ماتریس تجربی

کامل اطلاعات crosstalk و correlation را دارد.

۳. نویز مصنوعی کافی نیست: برای ارزیابی واقعی، باید حداقل از experimental_single و بهتر از

آن از experimental_correlated استفاده شود.

۴. مدل mitigation باید با شرایط آزمون سازگار باشد: در این گزارش فقط ارزیابی منطبق انجام

شده؛ train و test باید روی همان کانال نویز باشند.

جمع‌بندی نهایی

این پروژه یک مسیر کامل و قابل دفاع برای مطالعه نویز خوانش چهار کیوبیتی ساخته است (شکل ۱). در

بخش اول، داده‌های خام IQ با LDA به بیت‌های اندازه‌گیری شده تبدیل می‌شوند و پس از اعتبارسنجی،

ماتریس‌ها ذخیره می‌گردند (شکل‌های ۲ و ۳).

در بخش دوم، سه نوع نویز روی دو مدار کوانتومی اعمال می‌شود و دیتاست‌ها در DataTrain/<snapshot>

نگهداری می‌شوند. ارزیابی نهایی فقط با نویز منطبق train/test انجام می‌شود. مدل MLP نقش تقریب

کانال معکوس readout را دارد و با KL fidelity و L1 ارزیابی می‌شود (جدول ۵، شکل‌های ۶-۹ و

نمونه تک‌نمونه‌ای شکل ۱۰).

نتایج نشان می‌دهد mitigation در حالت منطبق بسیار مؤثر است: بیشترین بهبود fidelity ($\Delta F \approx$)

0.22 برای مدار مستقل و نویز همبسته است؛ در ZZ FeatureMap fidelity نهایی در هر سه نویز بالای

0.93 باقی می‌ماند. بازطراحی زیرساخت مبتنی بر snapshot پایه‌ای برای گسترش به داده تجربی بیشتر و

تبدیل این کار به مقاله علمی فراهم کرده است (جدول ۶).